

Encapsulamiento y Abstracción

Un sistema es una colección de componentes que interactúan entre sí con un objetivo común. En la construcción de sistemas complejos el **encapsulamiento** es un mecanismo fundamental porque permite utilizar cada componente como una “caja negra”, esto es, sabiendo **qué** hace sin saber **cómo** se hace. Se reducen así las dependencias entre las diferentes componentes, cada una de las cuales es más fácil de entender, probar y modificar.

El concepto de encapsulamiento está ligado a toda actividad vinculada a la producción. Una máquina compuesta se construye conectando varias máquinas simples, de modo que la salida de cada una constituye la entrada de otra y en conjunto alcanzan un propósito. Un motor, por ejemplo, se fabrica a partir de dispositivos mecánicos y energéticos. Estos dispositivos pueden usarse para construir otro tipo de máquinas. Un motor a su vez, va a ser usado como una componente en la construcción de otras máquinas.

Cada componente es un **módulo** que puede considerarse individualmente o como parte del conjunto. Cuando un módulo se analiza como parte de un todo, es posible concentrarse en qué función realiza, sin considerar cómo fue construido. Cuando un módulo se analiza individualmente es posible hacer abstracción de los detalles del contexto en el que va a ser utilizado, para enfocarse en cómo lograr su propósito.

En el desarrollo de sistemas de software el concepto de encapsulamiento está fuertemente ligado al de **abstracción**. Analistas y diseñadores elaboran un modelo a partir de un proceso de abstracción que especifica una colección de módulos relacionados entre sí. Los desarrolladores conservan en el código la estructura modular especificada en el diseño. Cada módulo puede ser verificado independientemente de los demás, antes de integrarse al sistema. Durante surgen cambios en la especificación o en el diseño, el encapsulamiento permite reducir el impacto.

La construcción de cada módulo de software puede realizarse sin conocer el sistema al cual va a integrarse. El sistema completo se construye a partir de una colección de módulos, sin considerar cómo se implementó cada uno en particular.

Encapsulamiento en la Programación Orientada a Objetos

En la programación orientada a objetos el concepto de encapsulamiento está ligado tanto a las clases como a los objetos. Cada objeto de software interactúa con otros objetos del entorno a través de su **interfaz** y **oculta** su estado interno y los detalles que describen cómo actúa cuando recibe un mensaje. Así, si la estructura del estado interno se modifica, el cambio no afecta a los otros miembros del entorno. Sólo las modificaciones en la interfaz pueden afectar a los demás objetos.

Para que un objeto de software oculte su estado interno, la clase que define sus atributos y comportamiento debe **esconder** la implementación. Cada clase es una caja negra, es decir, conoce **qué** hacen las demás clases del sistema, pero no **cómo** lo hacen.

El encapsulamiento¹ es entonces el mecanismo que permite **agrupar** en una clase los atributos y el comportamiento que caracterizan a sus instancias y **esconder** la representación de los datos y los algoritmos que modelan al comportamiento, de modo que sólo sean visibles dentro de la clase.

La **estructura estática** de un sistema orientado a objetos queda establecida por una colección de clases relacionadas entre sí. El **modelo dinámico** es un entorno poblado de objetos comunicándose a través de mensajes. El comportamiento de cada objeto en respuesta a un mensaje, depende de la clase a la que pertenece.

El encapsulamiento brinda cierto nivel de independencia, cada clase puede ser construida y verificada antes de integrarse al sistema completo. Una clase puede modificar la representación de los datos o los algoritmos, sin afectar a sus clases clientes, en tanto mantenga sus responsabilidades y los servicios conserven sus compromisos y funcionalidades.

Estructura de datos

Una **estructura de datos** es una colección de datos organizados de alguna manera. Los atributos de una clase definen una estructura de datos que permite representar el estado interno de los objetos de esa clase en ejecución. La programación orientada a objetos propone encapsular la representación de los datos, de modo que no sean visibles desde el exterior de la clase.

En Java los **modificadores de acceso** permiten establecer la visibilidad de los miembros de una clase. Los atributos se declaran privados y quedan así escondidos dentro de la clase.

Caso de Estudio: Ciudad

*El Ministerio de Educación de la Provincia ofrece un Programa de Capacitación Docente para maestros de nivel Primario y se decide desarrollar un sistema para la gestión del Programa. El Programa ofrece diferentes talleres para maestros. En cada **taller** se proponen **actividades** que luego los **maestros** desarrollan en la **escuela** en la que trabajan. El sistema de gestión requiere administrar la **inscripción** de los maestros en los talleres, el control de **asistencia**, la asignación de **capacitadores**, el registro de las **evaluaciones** y las escuelas. Al Ministerio le interesa realizar un análisis comparativo del Programa de Capacitación en cada **ciudad** y en cada **barrio**. El análisis considera la **población** de cada barrio y cada ciudad, la **densidad** de la ciudad, la **cantidad de alumnos** en escuelas públicas y en escuelas privadas, en cada barrio. El sistema de gestión para el Programa de Capacitación requiere entonces modelar al conjunto de ciudades, barrios, maestros, talleres, capacitadores, inscripciones, etc.*

El modelo completo va a incluir una clase para cada conjunto de entidades. A partir del modelo completo, es posible implementar cada clase en particular. El siguiente diagrama modela a la clase **Ciudad**:

¹ Algunos autores utilizan los términos *encapsulamiento* y *oscurecimiento de información* para referenciar a dos conceptos relacionados pero diferentes. El primero agrupa atributos y servicios, el segundo esconde la implementación.

Ciudad
<<Atributos de instancia>> CP: entero nombre:String poblacion:entero superficie:real
<<Constructor>> Ciudad(cod:entero) <<Comandos>> establecerNombre(n:String) establecerPoblacion(p:entero) establecerSuperficie(s:real) establecerDensidad(d:real) <<Consultas>> obtenerCP():entero obtenerNombre():String obtenerPoblacion():entero obtenerSuperficie():real obtenerDensidad():real toString():String
El código postal se establece en el momento de la creación y no puede modificarse.

- `establecerDensidad(d:real)` Modifica el valor del atributo `poblacion` con `d*superficie`. Requiere `superficie` mayor a 0.
- `obtenerDensidad():real` Computa `poblacion/superficie`. Requiere `superficie > 0`.

El diseñador decidió que es relevante incluir atributos para el código postal, el nombre, la población y la superficie. En la caracterización interesa también conocer la densidad de cada ciudad, pero el diseñador resolvió que la densidad no se mantenga como un atributo, sino que se calcule como el cociente entre la población y la superficie. También explicitó como responsabilidad de la clase cliente controlar que la superficie sea un valor positivo antes de enviar el mensaje `obtenerDensidad`.

Aunque también el código postal y la población deberían ser valores positivos, el diseñador no ha explicitado que estos valores deben ser validados ni por la clase `Ciudad` ni por sus clientes. Como programadores solo vamos a agregar controles en los datos que eviten la terminación anormal del programa. El resto de la validación va a modelar a las restricciones explícitas en la especificación.

De acuerdo a esta propuesta, en el momento que se crea una ciudad se establece únicamente el código, que no puede ser luego modificado. La clase brinda una consulta para obtener cada atributo y una más para obtener la densidad. Para las clases cliente, una ciudad tiene una densidad, más allá de que se mantenga en el estado interno o se compute.

La implementación de la clase `Ciudad` es:

```
public class Ciudad {
//Atributos de instancia
private int CP;
```

```

private int poblacion;
private float superficie;
private String nombre;
//Constructor
public Ciudad (int cod )
{ CP = cod; poblacion = 0 ; superficie = 0 ;}
//Comandos
public void establecerNombre (String n)
{ nombre = n ; }
public void establecerPoblacion (int p )
{ poblacion = p ; }
public void establecerSuperficie ( float s)
{ superficie = s ;}
public void establecerDensidad ( float d)
{/*Modifica el valor del atributo poblacion con d*superficie. Requiere
superficie mayor a 0.*/
    poblacion = (int) (d*superficie) ;}
//Consultas
public String obtenerNombre()
{ return nombre;}
public int obtenerCP ()
{ return CP; }
public int obtenerPoblacion ()
{ return poblacion; }
public float obtenerSuperficie ()
{ return superficie; }
public double obtenerDensidad ()
{/* Computa poblacion/superficie. Requiere superficie > 0
    return (double) poblacion/superficie; }
}

```

Los atributos se declaran como privados, quedan así escondidos dentro de la clase. Las clases clientes de `Ciudad` no pueden acceder directamente al estado interno de sus instancias.

La clase brinda cuatro comandos que permiten modificar el estado interno de un objeto de clase `Ciudad`. Si la clase cliente usa el servicio `establecerDensidad()` no se modifica el valor de este atributo, porque de hecho no se mantiene explícitamente, sino que se computa un nuevo valor para el atributo `poblacion`. El cómputo requiere conversión de tipos.

La clase `Ciudad` asume que sus clientes cumplen con sus responsabilidades. Si se envía el mensaje `obtenerDensidad()` a un objeto, antes de asignarse un valor a `superficie`, la ejecución terminará anormalmente. En un sistema **robusto**, cada clase debería prevenir situaciones de error, más allá de sus propias responsabilidades.

Consideremos que una vez implementado el sistema se resuelve modificar la representación interna de la clase `Ciudad`.

Ciudad
<<Atributos de instancia>> CP:entero nombre:String superficie:real densidad:real

La signatura de los métodos no cambia pero si la implementación:

- `establecerPoblacion(p:real)` Modifica el valor del atributo `densidad`, computando `p/superficie`. Requiere `superficie` mayor a 0.
- `obtenerPoblacion()` Computa la parte entera de la expresión `densidad*superficie`. Requiere `superficie` mayor a 0.

El código en Java de la clase `Ciudad` es ahora:

```
public class Ciudad {
//Atributos de instancia
private int CP;
private float densidad;
private float superficie;
private String nombre;
//Constructor
public Ciudad (int cod )
{ CP = cod; densidad = 0 ; superficie = 0 ;}
//Comandos
public void establecerNombre (String n)
{ nombre = n ; }
public void establecerPoblacion (int p )
{/* Modifica el valor del atributo densidad, computando p/superficie.
Requiere superficie mayor a 0.*/
    densidad = p/superficie ; }
public void establecerSuperficie (float s)
{ superficie = s ;}
public void establecerDensidad (float d)
{ densidad = d ;}
//Consultas
public String obtenerNombre()
{ return nombre;}
public int obtenerCP ()
{ return CP; }
public int obtenerPoblacion ()
{/* Computa la parte entera de la expresión densidad*superficie.
Requiere superficie mayor a 0. */
    return (int) (superficie*densidad); }
public float obtenerSuperficie ()
{ return superficie; }
public double obtenerDensidad ()
{ return densidad; } }
```

En la primera implementación de `Ciudad`, si los atributos fueran públicos, sus clientes podrían haber accedido directamente al atributo `poblacion`. Al cambiar el diseño de la clase proveedora, tendría que modificarse también cada una de las clases cliente. Siguiendo los lineamientos de la programación orientada a objetos, las clases clientes de `Ciudad` sólo acceden a su interfaz, que no cambió aun cuando se modificaron los atributos y la implementación de algunos servicios.

Representaciones alternativas para estructuras lineales y homogéneas

En la resolución de problemas de mediana y gran escala el diseño de las estructuras de datos es un tema relevante. Cuando una clase modela una secuencia homogénea de elementos, por lo general existen diferentes estructuras alternativas para representar tanto a los elementos como a la colección misma. Si cada servicio de la clase mantiene la signatura, además de los requisitos y funcionalidad, los cambios en la representación de los datos no modifican la interfaz, aunque sí va a ser necesario modificar el código.

Vamos a utilizar **arreglos** para modelar estructuras **lineales**, **homogéneas** y con **acceso directo**. Una estructura es **homogénea** si todos los elementos son del mismo tipo. En una estructura **lineal** cada elemento tiene un predecesor, excepto el primero, y un sucesor, excepto el último. Una estructura tiene **acceso directo** si cada una de sus componentes puede accederse a través de su posición, sin necesidad de acceder a las que la preceden o suceden.

Caso de Estudio: Estación Meteorológica

En una estación meteorológica se almacenan las temperaturas mínimas y máximas de cada día de un período y se computan algunos valores estadísticos.

Los servicios y las responsabilidades especificadas por el diseñador de la clase TempMinMaxEstación son:

```
<<Constructor>>
TempMinMaxEstacion(dias:entero)
<<Comandos>>
establecerTMin(dia:entero,t:real)
establecerTMax(dia:entero,t:real)
<<Consultas>>
obtenerTMin(dia:entero):real
obtenerTMax(dia:entero):real
contarHeladas():entero
contarTodoHeladas():entero
mayorMaxima():real
mayorAmplitud():real
huboMayAT(g:real):boolean
huboMayATCon(g:real):boolean
huboMayATnCon(g:real,n:entero):boolean
exactaNConAT(g:real,n:entero):boolean
cantMaxHeladas():entero
```

La entrada y salida se realizará desde la clase cliente, que además deberá validar que la máxima de cada día sea mayor o igual a la mínima. Antes de registrarse los valores se conoce la cantidad de días del período y todos los valores han sido almacenados en el momento que se computan estadísticas. El primer día de un período se denota con el número 1 y así correlativamente, el último día de un período con n días corresponde al número n. Los servicios que reciben un parámetro que corresponde al día requieren que este valor sea válido.

La funcionalidad de cada servicio de la clase `TempMinMaxEstacion` es:

- `TempMinMaxEstacion(dias:entero)` Requiere `dias` mayor a 0.
- `contarHeladas():entero` cuenta la cantidad de días del período en los que heló en algún momento del día.
- `contarTodoHeladas():entero` cuenta la cantidad de días en los que heló todo el día.
- `mayorMaxima():real` computa la mayor temperatura máxima.
- `mayorAmplitud():real` computa la mayor amplitud térmica del período.
- `huboMayAT(g:real):boolean` retorna `true` si y sólo si hubo al menos un día con amplitud térmica mayor a `g`.
- `huboMayATCon(g:real):boolean` retorna `true` si y sólo si hubo al menos dos días consecutivos con amplitud térmica mayor a `g`.
- `huboMayATnCon(g:real,n:entero):boolean` retorna `true` si y sólo si hubo al menos `n` días consecutivos con amplitud térmica mayor a `g`.
- `exactaNConAT(g:real,n:entero):boolean` retorna `true` si y sólo si hubo exactamente `n` días consecutivos con amplitud térmica mayor a `g`.
- `cantMaxHeladas():boolean` retorna la duración en días del período más largo con heladas en días consecutivos.

La implementación de cada método requiere diseñar un algoritmo que va a quedar escondido en la clase. En el diseño de cada uno seguimos los lineamientos de la programación estructurada.

Para representación de los datos la **primera alternativa** de diseño que proponemos es:

TempMinMaxEstacion	TMinMax
<<Atributos de instancia>> rt:TMinMax[]	<<Atributos de instancia>> min,max:real
	<<Constructor>> TMinMax (mi,ma:real) <<Comandos>> establecerMin(m:real) establecerMax(m:real) <<Consultas>> obtenerMin():real obtenerMax():real amplitudTermica():real equals(r:TMinMax):Boolean

Una **segunda alternativa** de diseño es:

TempMinMaxEstacion		TMinMax
<<Atributos de instancia>> rt: [] TMinMax		<<Atributos de instancia>> mm: [] real
		<<Constructor>> TMinMax (mi,ma:real) <<Comandos>> establecerMin(m:real) establecerMax(m:real) <<Consultas>> obtenerMin():real obtenerMax():real amplitudTermica():real equals(r:TMinMax):Boolean

En estos dos diagramas la clase **TempMinMaxEstación** está asociada a la clase **TMinMax**. Es decir, las componentes de la estructura de datos son también estructuras de datos. Las responsabilidades establecidas para los servicios de **TMinMax** son en ambos casos:

- **TMinMax (mi,ma:real)** Requiere $ma \geq mi$
- **establecerMin(m:real)** Requiere $m \leq max$
- **establecerMax(m:real)** Requiere $m \geq min$

Cada clase conforma un módulo o unidad de programación, a partir de los cuales se crean objetos y se les envían mensajes, sin conocer su implementación. El cambio en **TMinMax** no afecta **TempMinMaxEstacion**. En los dos casos la clase **TMinMax** asume que se han controlado previamente los valores de las temperaturas máxima y mínima de modo tal que la primera es mayor o igual que la segunda. Puede haberlo hecho la clase asociada o cualquiera de las clases que la usan.

En la siguiente alternativa, la clase **TempMinMaxEstacion** esconde a dos estructuras de datos, cuyas componentes son valores elementales. El **tercer diseño** es:

TempMinMaxEstacion
<<Atributos de Instancia>> min: [] real max: [] real

La representación de los datos es transparente para las clases relacionadas con **TempMinMaxEstacion**. De hecho, es posible elegir una de las tres alternativas propuestas y luego cambiarla por otra de las representaciones, sin afectar a las clases cliente, ya que no cambia la interfaz ni las responsabilidades.

Como el primer día del período se referencia con 1, si las temperaturas se almacenan en un arreglo, va a ser necesario establecer un mapeo entre el día y el subíndice. Así, el día uno se almacena en la componente del arreglo que corresponde al subíndice 0, el día 2 en el subíndice 1 y así siguiendo.

Patrones de diseño de algoritmos

Con frecuencia los recorridos sobre las estructuras de datos responden a **patrones de algoritmos**, independientes del tipo de los elementos de la estructura. Por ejemplo, es habitual recorrer una estructura de datos lineal y homogénea para contar la cantidad de elementos que satisfacen una propiedad, el algoritmo puede ser entonces:

```
Algoritmo contar
Dato de Salida: contador
contador se inicializa en 0
para cada elemento de la estructura
    si el elemento cumple la propiedad
        incrementar el contador
```

El algoritmo `contar` describe el **comportamiento abstracto** de los servicios `contarHeladas` y `contarTodoHeladas`. El algoritmo puede **refinarse** en versiones más específicas:

<pre>Algoritmo contarHeladas Dato de salida: contador contador se inicializa en 0 para cada día del período si temperatura minima <= 0 incrementar el contador</pre>	<pre>Algoritmo contarTodoHeladas Dato de salida: contador contador se inicializa en 0 para cada día del período si temperatura maxima <= 0 incrementar el contador</pre>
---	---

Los algoritmos que implementan los servicios provistos por una clase están **escondidos** dentro de la clase. Si se modifican, en tanto no cambie la signatura, la funcionalidad ni los requisitos, la modificación es **transparente** para las clases cliente. Por ejemplo, el cómputo de la cantidad de componentes que verifican una propiedad dentro de un período, puede plantearse recursivamente:

Caso trivial: En un período vacío la cantidad de componentes que verifican una propiedad es 0.

Caso recursivo: En un período no vacío, la cantidad de componentes que verifican una propiedad es la cantidad de componentes que verifican la propiedad en el período que resulta de no considerar el último día, más 1 si el último día verifica la propiedad.

Caso recursivo: En un período no vacío, la cantidad de componentes que verifican una propiedad es la cantidad de componentes que verifican la propiedad en el período que resulta de no considerar el último día, si el último día NO verifica la propiedad.

Las clases relacionadas con `TempMinMaxEstacion` *no saben* si `contarHeladas` se implementa a partir de un algoritmo iterativo o un planteo recursivo.

Otro recorrido habitual sobre una estructura consiste en hallar el mayor elemento, de acuerdo a una relación de orden establecida. En este caso el algoritmo es:

```
Algoritmo mayor
Dato de salida: mayor
mayor ← primer elemento
para cada elemento de la estructura a partir del segundo
    si elemento > mayor
        mayor ← elemento
```

El algoritmo propone un recorrido exhaustivo, todos los elementos de la estructura deben ser considerados para computar el mayor. En cada iteración se accede a un elemento particular, comenzando por el segundo. Nuevamente este patrón general es una **abstracción**, que puede refinarse para cada problema específico:

<pre> Algoritmo mayorAmplitud Dato de salida: mayor mayor ← amplitud del primer día para cada día del período si amplitud del día > mayor mayor ← Amplitud del día </pre>	<pre> Algoritmo mayorMaxima Dato de salida: mayor mayor ← máxima del primer día para cada día del período si la máxima del día > mayor mayor ← máxima del día </pre>
--	---

Observemos que estamos utilizando una notación informal, en contraposición a la sintaxis rigurosa de un lenguaje de programación. Podemos usar el símbolo $\leftarrow$ para denotar asignación o utilizar una notación verbal como “inicializar mayor con la amplitud térmica del primer día” o “asignar la amplitud térmica del primer día a mayor”.

Para decidir si algún elemento de la estructura verifica una propiedad no es necesario hacer un recorrido exhaustivo:

```

Algoritmo verifica
Dato de salida: verifica
verifica ← falso
para cada elemento de la estructura y mientras no verifica
    si el elemento actual verifica la propiedad
        verifica ← verdadero

```

En el caso específico de decidir si la amplitud térmica fue mayor a g en algún día del período, el patrón se refina:

```

Algoritmo huboMayAT
Dato de entrada: g
Dato de salida: verifica
verifica ← falso
para cada día del período y mientras no verifica
    si la amplitud térmica del día > g
        verifica ← verdadero

```

El mismo patrón puede refinarse un poco más para modelar la solución de decidir si hubo dos días seguidos con amplitud térmica mayor a un valor g .

```

Algoritmo huboMayATCon
Dato de entrada: g
Dato de salida: verifica
verifica ← falso
para cada día del período excepto el último y mientras no verifica
    si la amplitud térmica del día > g y
        la amplitud térmica del día siguiente > g
        verifica ← verdadero

```

El algoritmo para decidir si hubo al menos n días consecutivos con temperaturas mayores a g , implica nuevamente un recorrido no exhaustivo:

```

Algoritmo huboMayATnCon
Dato de entrada:  g,n
Dato de salida:  verifica
inicializa contador en 0
para cada día del período y mientras contador < n
    si la amplitud térmica del día > g
        incrementa el contador
    sino inicializa contador en 0
verifica es verdadero si contador = n

```

Podemos refinar el algoritmo anterior considerando que la búsqueda puede interrumpirse antes si podemos asegurar que la propiedad no va cumplirse. En particular, si el período tiene 15 días y buscamos 16 días consecutivos con temperaturas mayores a g, sin recorrer la estructura podemos asegurar que no hay n consecutivos.

Si el problema plantea decidir si hubo exactamente n días que verifiquen la propiedad:

```

Algoritmo exactaNConAT
Dato de entrada:  g,n
Dato de salida:  verifica
inicializa contador en 0
para cada día del período y mientras contador <= n
    si la amplitud térmica del día > g
        incrementa el contador
    sino inicializa contador en 0
verifica es verdadero si contador = n

```

Implementaciones alternativas

En los dos primeros diseños alternativos propuestos, las clases `TempMinMaxEstacion` y `TMinMax` están **asociadas**. En ambos casos la implementación parcial de `TempMinMaxEstacion` a partir de los algoritmos propuestos, puede ser:

```

class TempMinMaxEstacion {
/* La entrada y salida se realizará desde la clase cliente, que
además deberá validar que la máxima de cada día sea mayor o igual a
la mínima. Antes de registrarse los valores se conoce la cantidad
de días del período y todos los valores han sido almacenados en el
momento que se computan estadísticas. El primer día de un período
se denota con el número 1 y así correlativamente, el último día de
un período con n días corresponde al número n. Los servicios que
reciben un parámetro que corresponde al día requieren que este
valor sea válido.*/
//Atributo de instancia
    private TMinMax [] rt;
//Constructor
public TempMinMaxEstacion (int dias){
/* Requiere días mayor a 0. Cada elemento del arreglo representa un día
del período*/
    rt = new TMinMax [dias];
    for (int dia = 0; dia < dias; dia++)
        rt[dia] = new TMinMax(0,0);}
//Comandos y Consultas triviales
public void establecerTempMin (int dia, float t){

```

```

    rt[dia-1].establecerMin(t);}
public void establecerTempMax (int dia, float t){
    rt[dia-1].establecerMax(t);}
public float obtenerTempMin (int dia){
    return rt[dia-1].obtenerMin();}
public float obtenerTempMax (int dia){
    return rt[dia-1].obtenerMax();}
//Consultas
public int cantDias(){
    return rt.length;}
public int cantHeladas(){
    /*Calcula la cantidad de días con temperatura mínima menor a 0*/
    int cant=0;
    for (int dia=0;dia<cantDias();dia++){
        if (rt[dia].obtenerMin()<0) cant++;
    }
    return cant;}
public float mayorAmplitud(){
    /*Computa el la mayor amplitud térmica del período*/
    float mayor = rt[0].amplitudTermica();
    float m;
    for (int dia=1;dia<cantDias();dia++){
        m = rt[dia].amplitudTermica();
        if (m > mayor) mayor = m;}
    return mayor;}}

```

La responsabilidad establecida en el diagrama se especifica también como un comentario al iniciarse la implementación. En este caso entonces, no hemos repetido los compromisos de la clase cliente en cada servicio que recibe como parámetro a una temperatura o un día.

La clase `TempMinMaxEstacion` se implementa sin conocer el código de `TMinMax`, sólo conocemos la interfaz de esta clase, especificada en el diagrama. Para el primer diagrama de `TMinMax`:

TminMax
min,max:real

La implementación puede ser:

```

class TMinMax{
//Requiere max >= min
//Atributos de instancia
private float min;
private float max;
//Constructor
public TMinMax (float mi,float ma){
//Requiere ma >= mi
    min = mi;
    max = ma;}
//Comandos
public void establecerMin(float m){
//Requiere m <= max
    min = m;}
public void establecerMax(float m){
//Requiere m >= min

```

```

    max = m;}
//Consultas
public float obtenerMin(){
    return minima;}
public float obtenerMax(){
    return max;}
public float amplitudTermica(){
    return max-min;}
public boolean equals(TMinMax r){
    return min == r.obtenerMin() &&
           max == r.obtenerMax();}}

```

Consideremos ahora el **segundo diseño**, en el cual se modifica la clase `TMinMax`:

TMinMax
mm:[] real

La implementación puede ser:

```

class TMinMax{
//Requiere max >= min
//Atributos de instancia
private float [] mm;
//Constructor
public TMinMax (float mi,float ma){
//Requiere ma >= mi
    mm = new float [2];
    mm[0] = mi;
    mm[1] = ma;}
//Comandos
public void establecerMin(float m){
//Requiere m <= max
    mm[0] = m;}
public void establecerMax(float m){
//Requiere m >= min
    mm[1] = m;}
//Consultas
public float obtenerMin(){
    return mm[0];}
public float obtenerMax(){
    return mm[1];}
public float amplitudTermica(){
    return mm[1]-mm[0];}
public boolean equals(TMinMax r){
    return mm[0] == r.obtenerMin() &&
           mm[1] == r.obtenerMax();}}

```

Notemos que el cambio no tendría impacto en la clase `TempMinMaxEstacion`, ya que esta clase no accede directamente a los atributos de `TMinMax`.

Los atributos de instancia en la tercera alternativa de diseño de `TempMinMaxEstacion` son:

TempMinMaxEstacion
min:[] real
max:[] real

En este caso sí es necesario modificar el código de `TempMinMaxEstacion`. La implementación parcial es entonces:

```
class TempMinMaxEstacion {
/* La entrada y salida se realizará desde la clase cliente, que además
deberá validar que la máxima de cada día sea mayor o igual a la
mínima. Antes de registrarse los valores se conoce la cantidad de días
del período y todos los valores han sido almacenados en el momento que
se computan estadísticas. El primer día de un período se denota con el
número 1 y así correlativamente, el último día de un período con n
días corresponde al número n. Los servicios que reciben un parámetro
que corresponde al día requieren que este valor sea válido.*/
//Atributo de instancia
    private float [] min;
    private float [] max;
//Constructor
public TempMinMaxEstacion (int dias){
/*Requiere días mayor a 0. Cada elemento del arreglo representa un día
del período*/
    min = new float [dias];
    max = new float [dias];}
//Comandos y Consultas triviales
public void establecerTempMin (int dia, float t){
    min[dia-1] = t;}
public void establecerTempMax (int dia,float t){
    max [dia-1] = t;}
public float obtenerTempMin (int dia){
    return min[dia-1];}
public float obtenerTempMax (int dia){
    return max[dia-1];}
//Consultas
public int cantDias(){
    return min.length;}
public int cantHeladas(){
//Calcula la cantidad de días con temperatura mínima menor a 0
    int cant=0;
    for (int dia=0;dia<cantDias();dia++)
        if (min[dia]<0) cant++;
    return cant;}
public float mayorAmplitud(){
//Computa la mayor amplitud térmica del período
    float mayor = max[0]-min[0];
    float m;
    for (int dia=1;dia<cantDias();dia++){
        m = max[dia]-min[dia];
        if (m > mayor)
            mayor = m;}
    return mayor;} }
```

A pesar del cambio de representación de los datos, las dos implementaciones corresponden a los mismos patrones de diseño para los algoritmos.

Encapsulamiento y clases relacionadas

La modificación de la representación de los datos en una clase implica reescribir el código de algunos o incluso todos los servicios. Si los atributos están escondidos, el cambio en la representación es transparente para las clases asociadas, en tanto no cambien las responsabilidades de la clase ni la funcionalidad o los requisitos de los servicios.

Las clases clientes de `TempMinMaxEstacion` sólo pueden acceder a los atributos a través de los servicios provistos en la interfaz. Si cambia la representación, el cambio no impacta sobre las clases relacionadas.

En particular, es posible implementar una misma clase tester para verificar las dos versiones de `TempMinMaxEstacion`. La clase `TestTempMinMaxEstacion` verifica el método `mayorAmplitud` con valores fijos:

```
class TestTempMinMaxEstacion {
public static void main(String[] args) {
/* Verifica el método mayorAmplitud para tres casos de prueba
significativos*/
    TempMinMaxEstacion est;
    int cantD =7;
    est =genTempMinMaxEst(cantD) ;
    System.out.println("Muestra la estación  ");
    mostrarTempMinMaxEst(est) ;
    System.out.println(" La mayor amplitud es "+est.mayorAmplitud());
    //Caso de prueba: La mayor amplitud se produjo en el primer día
    est.establecerTempMin(1,10);
    est.establecerTempMax(1,25);
    System.out.println("Muestra la estación  ");
    mostrarTempMinMaxEst(est) ;
    System.out.println(" La mayor amplitud es "+est.mayorAmplitud());
    //Caso de prueba: La mayor amplitud se produjo el último día
    est.establecerTempMin(est.cantDias(),10);
    est.establecerTempMax(est.cantDias(),30);
    System.out.println("Muestra la estación  ");
    mostrarTempMinMaxEst(est) ;
    System.out.println(" La mayor amplitud es "+est.mayorAmplitud());}
public static TempMinMaxEstacion genTempMinMaxEst(int cantD){
    TempMinMaxEstacion e;
    e=new TempMinMaxEstacion(cantD);
    for (int dia= 1; dia<e.cantDias()-1; dia++){
        e.establecerTempMin(dia,-1);
        e.establecerTempMax(dia,dia+8);}
    e.establecerTempMin(6,5);
    e.establecerTempMax(6,15);
    e.establecerTempMin(e.cantDias(),5);
    e.establecerTempMax(e.cantDias(),11);
    return e;}
public static void mostrarTempMinMaxEst( TempMinMaxEstacion est){
    for (int dia=1; dia<=est.cantDias(); dia++)
        System.out.println(" "+est.obtenerTempMin(dia)+
            " "+est.obtenerTempMax(dia) );}
}
```

Esta clase tester considera un período de 7 días y tres casos de prueba: la mayor amplitud térmica se produce el primer día, el último y uno cualquiera que no es ni el primero ni el último. Tanto si los valores están establecidos en la clase tester, son leídos por consola o desde un archivo, todos los datos están almacenados en el arreglo antes de que se realice cualquier procesamiento.

La verificación de servicios como `huboMayATnCon(g:real,n:entero):boolean` va a requerir definir un mayor número de casos de prueba:

- todos los elementos tienen amplitud mayor a g, pero son menos de n
- la estructura tiene más de n elementos, pero no hay n con amplitud térmica mayor a g.
- los primeros n-1 elementos de la estructura tienen amplitud térmica mayor a g, el siguiente es igual a g y luego sigue otro con amplitud mayor a g.
- los primeros n-1 elementos de la estructura tienen amplitud térmica mayor a g, el siguiente es menor a g y luego sigue otro con amplitud mayor a g.
- los primeros n elementos de la estructura tienen amplitud térmica mayor a g.
- solo los últimos n elementos de la estructura tienen amplitud térmica mayor a g.

Tengamos en cuenta que mientras implementamos la clase `TempMinMaxEstacion` buscamos que funcione correctamente, cuando implementamos la clase tester el objetivo es identificar los casos para los cuales `TempMinMaxEstacion` NO funciona correctamente.

Abstracción y programación orientada a objetos

El análisis y diseño orientado a objetos demanda un proceso de abstracción a partir del cual se identifican los objetos del problema y se agrupan en clases. Una clase es una abstracción, un patrón que caracteriza los atributos y el comportamiento de un conjunto de objetos.

En la implementación, una clase que establece atributos y servicios define un **tipo de dato** a partir del cual es posible crear instancias. El conjunto de valores queda determinado por el tipo de los atributos y el conjunto de operaciones corresponde a los servicios provistos por la clase. Si la representación de los datos está escondida, la clase define un **tipo de dato abstracto (TDA)**.

La definición de tipos de datos abstractos es un recurso importante porque favorece la reusabilidad y permite reducir el impacto de los cambios. La modificación de una clase que define un tipo de dato abstracto, puede realizarse sin afectar a las clases que crean instancias del tipo.

En Java la definición de clases y los modificadores de accesos, permiten que el programador defina nuevos tipos de datos abstractos a partir de los cuales se crean instancias. Todas las clases implementadas en los casos de estudio propuestos hasta el momento definen tipos de datos abstractos. A continuación presentamos varios casos de estudio, cada uno de los cuales define TDA asociados.

TDA y clases relacionadas

El siguiente caso de estudio ilustra como un cambio en la implementación del TDA `Circulo`, es transparente para las clases relacionadas por asociación o dependencia.

Caso de Estudio: Círculo y Punto

Un sistema de software didáctico de geometría para preescolar brinda facilidades para que los niños clasifiquen figuras geométricas. En el desarrollo del sistema el diseñador ha elaborado un diagrama con más de 50 clases, entre ellas `Circulo` y `Punto`, definidos como TDA.

Punto	Circulo
<pre><<Atributos de instancia>> x,y:real</pre>	<pre><<Atributos de clase>> Pi:real <<Atributos de instancia>> radio:real centro:Punto</pre>
<pre><<Constructor>> Punto(x,y:real) <<Comandos>> copy (p:Punto) establecerX(x:real) establecerY(y:real) <<Consultas>> obtenerX():real obtenerY():real equals(p:Punto):boolean distancia(p:Punto):real clone():Punto toString():String</pre>	<pre><<Constructor>> Circulo(r:real,p:Punto) <<Comandos>> trasladar(p:Punto) escalar(r:real) copy (c:Circulo) <<Consultas>> obtenerCentro():Punto obtenerRadio():real equals(c:Circulo):boolean area():real perimetro():real clone():Circulo</pre>

En la clase `Punto`:

- `copy(p:Punto)` Requiere `p` ligado.
- `equals(p:Punto):boolean` Requiere `p` ligado.
- `distancia(p:Punto):real` computa la distancia entre el punto que recibe el mensaje y el punto que pasa como parámetro, requiere `p` ligado.

En la clase `Circulo`:

- `Circulo(r:real,p:Punto)` Requiere `p` ligado y `r` mayor a 0.
- `trasladar(p:Punto)` Requiere `p` ligado.
- `escalar(r:real)` Requiere `r` mayor a 0.
- `copy(c:Circulo)` Requiere `c` ligado, se implementa en forma superficial.
- `equals(c:Circulo):boolean` Requiere `c` ligado, se implementa en forma superficial.
- `clone():Circulo` se implementa en forma superficial.

Las clases `Circulo` y `Punto` están relacionadas por asociación. La clase `Circulo` tiene un atributo de clase `Punto`. El constructor de `Circulo` y el método `trasladar` reciben un parámetro de clase `Punto`. El método `obtenerCentro` retorna como resultado un objeto de clase `Punto`. Existe también una relación de dependencia entre `Circulo` y `Punto`.

La implementación de la clase `Punto` es:

```
class Punto{
//Atributos de Instancia
private float x,y;
//Constructores
public Punto (float x,float y){
    this.x = x;  this.y = y;}
//Comandos
public void establecerX (float x){
    this.x = x;}
public void establecerY (float y){
    this.y = y;}
public void copy (Punto p){
//Requiere p ligada
    x = p.obtenerX();
    y = p.obtenerY();}
//Consultas
public float obtenerX(){ return x;}
public float obtenerY(){ return y;}
public double distancia(Punto p) {
//Requiere p ligada
    float dx= x - p.obtenerX();
    float dy= y - p.obtenerY();
    return Math.sqrt(dx*dx+dy*dy);}
public boolean equals (Punto p){
//Requiere p ligada
    return x==p.obtenerX() && y==p.obtenerY();}
public Punto clone(){
    return new Punto(x,y);}
public String toString(){
    return "("+x+","+y+")";}}
```

La clase `Punto` define un tipo de dato abstracto a partir del cual es posible crear instancias. El conjunto de valores es el conjunto de pares de valores de tipo `float`. Cada servicio provisto por la clase corresponde a una operación provista por el tipo.

La clase `Circulo` mantiene un atributo de instancia del TDA `Punto`, como ilustra la siguiente implementación:

```
class Circulo {
//El centro está siempre ligado
//Atributos de clase
private static final float Pi=3.1415f;
//Atributos de Instancia
private float radio;
private Punto centro;
//Constructor
public Circulo(float r, Punto p){
//Requiere p ligada y r mayor a 0
    radio = r;
    centro = p;}
//Comandos
public void trasladar(Punto p){
//Requiere p ligada
    centro = p;}
public void escalar(float r){
```

```
//Requiere r mayor a 0
    radio = r;}
public void copy (Circulo c){
//Requiere c ligada
    radio = c.obtenerRadio();
    centro = c.obtenerCentro();}
//Consultas
public Punto obtenerCentro(){
    return centro;}
public float obtenerRadio(){
    return radio;}
public float perimetro(){
    return obtenerRadio()*2*Pi;}
public double area(){
    return Pi * obtenerRadio()* obtenerRadio();}
public boolean equals(Circulo c){
//Requiere c ligada
    return obtenerRadio()== c.obtenerRadio() &&
        obtenerCentro() == c.obtenerCentro();}
public Circulo clone (){
    return new Circulo (obtenerRadio(),obtenerCentro());}}
```

Si `centro` debe estar siempre ligada, tanto el constructor como el método `trasladar` reciben la variable `p` ligada.

La clase `Circulo` también define un tipo de dato abstracto a partir del cual es posible crear instancias. La clase `Circulo` es cliente de la clase `Punto` y a su vez es proveedora de servicios para las clases que la usan.

Consideremos ahora el siguiente diseño alternativo para la clase `Circulo`:

Circulo
<<Atributos de instancia>> centro:Punto punto:Punto
<<Constructor>> Circulo(r:real,p:Punto) <<Comandos>> trasladar(p:Punto) escalar(r:real) copy (c:Circulo) <<Consultas>> obtenerCentro():Punto obtenerRadio():real equals(c:Circulo):boolean area():real perimetro():real clone():Circulo

En este diseño, un objeto de clase `Circulo` se modela a través de dos atributos de instancia que corresponden al TDA `Punto`. Uno corresponde al centro y otro a un punto cualquiera de la circunferencia. La funcionalidad y los requisitos de los servicios no cambian. El cambio en la representación tampoco modifica la signatura de los servicios. Por lo tanto, no se modifica la interfaz de la clase. Como los atributos están escondidos, el cambio no afecta a la clase `Punto`, proveedora de la clase `Circulo`, ni a las clases clientes de la clase `Circulo`.

La modificación en el diseño impacta en la implementación, el código de varios servicios tiene que cambiar, aun cuando la funcionalidad y los requisitos no hayan variado. El constructor de `Circulo` se implementa ahora como:

```
public Circulo (float r, Punto p){
//Requiere p ligada y r mayor a 0
    centro = p;
    punto = new Punto (centro.obtenerX(), centro.obtenerY()+r);}
```

El método `obtenerRadio()` se modifica porque el radio ya no es un atributo de la clase `Circulo`.

```
public float obtenerRadio(){
    return centro.distancia(punto);}
```

Las clases dependientes o asociadas a la clase `Circulo` no perciben el cambio de diseño ni de la implementación porque sólo acceden a la interfaz de la clase proveedora.

TDA y estructuras homogéneas, lineales y con acceso directo

El diseño de una estructura de datos está fuertemente ligado a cómo van a accederse las componentes. Los arreglos son adecuados para representar estructuras de datos homogéneas y lineales, cuyas componentes deben ser accedidas directamente a través de su posición.

Caso de Estudio: TDA Matriz-Vector

Un sistema de software didáctico de álgebra elemental brinda facilidades para que los jóvenes operen con matrices y vectores. En el desarrollo del sistema el diseñador ha elaborado un diagrama con más de 50 clases, entre ellas Matriz y Vector, definidos como TDA. Implementar los TDA `Matriz` y `Vector` a partir del siguiente diagrama:

Matriz	Vector
<<Atributos de instancia>> mr:[] [] real	<<Atributos de instancia>> v:[] real
<<Constructor>> Matriz(nfil,ncol:entero) <<Comandos>> establecerElem(f,c:entero, elem:real) copy(m:Matriz) establecerIdentidad() invertirFilas(f1,f2:entero) xEscalar(r:real) <<Consultas>> existePos(f,c:entero):boolean obtenerNfil():entero obtenerNcol():entero obtenerElem(f,c:entero):real clone():Matriz equals(m:Matriz):boolean esCuadrada():boolean esIdentidad():boolean esTriangularSuperior():boolean esSimetrica():boolean	<<Constructor>> Vector(n:entero) <<Comandos>> establecerElem(f:entero, elem:real) copy(m:Vector) xEscalar(r:real) <<Consultas>> existePos(f:entero):boolean obtenerN():entero obtenerElem(f:entero):real clone():Vector equals(m:Vector):boolean mayorElemento():real productoEscalar(m:Vector):real productoVectorial(m:Vector): Vector

```

esRala():boolean
cantElem(elem:real):entero
mayorElemento():real
filaMayorElemento():entero
vectorMayores():Vector
suma(m:Matriz):Matriz
producto(m:Matriz):Matriz
transpuesta():Matriz

```

Todos los servicios que reciben objetos como parámetros requieren referencias ligadas.

Todos los servicios que reciben objetos como parámetros requieren referencias ligadas.

La clase `Matriz` depende de la clase `Vector` porque el servicio `vectorMayores()` retorna un objeto de clase `Vector`.

- `Matriz(nfil,ncol:entero)` requiere `nfil` y `ncol` mayores a 0.
- `establecerElem(f,c:entero, elem:real)` Requiere `0<=f<obtenerNfil()` y `0<=c<obtenerNcol()`.
- `copy(m:Matriz)` Crea un nuevo objeto para el atributo de instancia con el nro de filas y columnas de `m` y luego realiza la copia.
- `establecerIdentidad()` Requiere `obtenerNcol()` igual a `obtenerNcol()`.
- `invertirFilas(f1,f2:entero)` Requiere `0<=f1,f2 <obtenerNfil()`. Si `f1` y `f2` son iguales, el comando no provoca ningún cambio en el estado interno del objeto.
- `obtenerElem(f,c:entero):` Requiere `0<=f <obtenerNfil()` y `0<=c <obtenerNcol()`.
- `clone():Matriz` Implementa clone.
- `equals(m:Matriz): boolean`
- `vectorMayores():Vector` Genera un objeto de clase `Vector` en el cual el elemento `i` es el mayor de la fila `i` de la matriz.

El método `xEscalar` implementa el siguiente algoritmo:

```

Algoritmo xEscalar
Datos de Entrada: r
para cada posición i,j
    mri,j = mri,j*r

```

En el método `copy`, `m` es un objeto de clase `Matriz` y `mr` es un arreglo de dos dimensiones. Para acceder a los elementos de `m` se usan los servicios provistos por la clase, mientras que los elementos de `mr` se acceden con subíndices. Aunque la especificación de este comando no indica específicamente que requiere el parámetro ligado, en las responsabilidades de la clase el diseñador estableció que todos los servicios que reciben objetos como parámetros requieren referencias ligadas, de modo que `copy` es un caso particular.

Un planteo recursivo para decidir si una matriz es la matriz identidad puede ser:

Caso trivial

Una matriz **mr** de 1x1 es la matriz identidad si su único elemento **mr[0,0]** es 1

Caso recursivo

Una matriz **mr** de nxn con $n > 1$ es la matriz identidad si

mr[i,n-1] = 0 para $0 \leq i < n-1$

mr[n-1,j] = 0 para $0 \leq j < n-1$

mr[n-1,n-1] = 1

y la matriz **mr'** de (n-1)x(n-1) es la matriz identidad

La implementación parcial de la clase **Matriz** es:

```
class Matriz {
/*Todos los servicios que reciben objetos como parámetros requieren
referencias ligadas.*/
private float[][] mr;
//Constructor
public Matriz(int nfil,int ncol){
//Requiere nfil y ncol mayores a 0
mr=new float[nfil][ncol];}
//Consultas
public int obtenerNFil()    {
return mr.length; }
public int obtenerNCol()    {
return mr[0].length; }
public boolean existePos(int f, int c) {
return(f>=0 && f < obtenerNFil()) &&
(c>=0 && c < obtenerNCol());}
public float obtenerElem(int f,int c){
//Requiere f y c consistentes
return mr[f][c];}
//Comandos
public void establecerElemento (int f,int c,
float x) {
//Requiere f y c consistentes
mr[f][c]= x;}
public void establecerIdentidad () {
//Requiere que el número de filas sea igual al de columnas
iniMatriz();
for (int j=0;j<obtenerNCol();j++)
mr[j][j] = 1; }
private void iniMatriz() {
for (int i=0;i<obtenerNFil();i++)
for (int j=0;j<obtenerNCol();j++)
mr[i][j] = 0; }
public void xEscalar (float r) {
//Multiplica cada elemento de la matriz por r
for (int i=0;i<obtenerNFil();i++)
for (int j=0;j<obtenerNCol();j++)
mr[i][j] = mr[i][j] * r;}
public void copy (Matriz m) {
/*Crea un nuevo objeto para el atributo de instancia con el nro de
filas y columnas de m */
mr = new float[m.obtenerNFil()][m.obtenerNCol()];
for (int i=0;i<obtenerNFil();i++) {
for (int j=0;j<obtenerNCol();j++)
```

```

        mr[i][j] = m.obtenerElem(i,j); } }
public int cantElem (float ele) {
    int cant = 0;
    for (int i=0;i< obtenerNFil();i++)
        for (int j=0;j< obtenerNCol();j++)
            if (mr[i][j] == ele) cant++;
    return cant;}
public boolean esCuadrada () {
    return (obtenerNFil() == obtenerNCol()) ; }
public boolean esIdent () {
    if (!esCuadrada())return false;
    else return esIdentidad(obtenerNFil());}
private boolean esIdentidad (int n) {
    if (n == 1)return (mr[0][0] == 1);
    else return (mr[n-1][n-1] == 1 &&
        esCeroFila(n-1) &&
        esCeroColumna(n-1)&&
        esIdentidad(n-1));}

```

La verificación de los TDA **Matriz** y **Vector** requiere como siempre establecer un conjunto de casos de prueba para cada operación provista por el tipo. Los servicios de la clase **Matriz** asumen que cada clase cliente cumple con los compromisos establecidos en el contrato. Cuando no basta que una solución sea correcta sino que se pretende que el sistema sea robusto, cada clase puede prevenir errores provocados por el incumplimiento de los compromisos de las clases clientes. Una manera de implementar el mecanismo de prevención es el manejo de excepciones.

Gran parte del código de la clase **Matriz** es independiente del tipo de las componentes, de modo que puede adaptarse o incluso reusarse, para modelar matrices cuyas componentes son objetos.

Caso de Estudio TDA Racional MatrizRacional y VectorRacional

Implementar los TDA **Racional**, **MatrizRacional** y **VectorRacional** definidos a partir del siguiente diagrama:

MatrizRacional	VectorRacional
<<Atributos de instancia>> mr:[] [] Racional	<<Atributos de instancia>> v:[] Racional
<<Constructor>> MatrizRacional(nfil,ncol:entero) <<Comandos>> establecerElem(f,c:entero, r:Racional) copy(m:MatrizRacional) establecerIdentidad() invertirFilas(f1,f2:entero) xEscalar(r:Racional) <<Consultas>> existePos(f,c:entero):boolean obtenerNFil():entero obtenerNCol():entero obtenerElem(f,c:entero):Racional	<<Constructor>> VectorRacional(n:entero) <<Comandos>> establecerElem(f:entero, r:Racional) copy(m:Racional) xRacional(r:Racional) <<Consultas>> existePos(f:entero): boolean obtenerN():entero obtenerElem(f:entero): Racional clone():VectorRacional

```

clone():MatrizRacional
equals(m:MatrizRacional):boolean
esCuadrada():boolean
esIdentidad():boolean
esTriangularSuperior():boolean
esSimetrica():boolean
esRala():boolean
cantElem(r:Racional):entero
mayorElemento():Racional
filaMayorElemento():entero
vectorMayores():VectorRacional
suma(m:MatrizRacional):
MatrizRacional
producto(m:MatrizRacional):
MatrizRacional
transpuesta():MatrizRacional

```

Cada elemento de la matriz es una referencia ligada cuando se ejecuta cualquier consulta, con excepción de existePos, obtenerNfil, obtenerNcol y obtenerElem. Todos los servicios que reciben a una matriz como parámetro requieren que la referencia esté ligada.

```

equals(m:VectorRacional):
boolean
mayorElemento():Racional
productoEscalar
(m:VectorRacional):Racional
productoVectorial(m:Vector):
Racional

```

Todos los servicios que reciben objetos como parámetros requieren referencias ligadas.

- `MatrizRacional(nfil,ncol:entero)` Requiere `nfil` y `ncol` mayores a 0.
- `establecerElem(f,c:entero, r:Racional)` Requiere `0<=f <obtenerNfil()` y `0<=c <obtenerNcol()`. Requiere `r` ligado.
- `establecerIdentidad()` Requiere `obtenerNcol()` igual a `obtenerNcol()`.
- `invertirFilas(f1,f2:entero)` Requiere `0<=f1,f2<obtenerNfil()`. Si `f1` y `f2` son iguales, el comando no provoca ningún cambio.
- `copy(m:MatrizRacional)` Liga un nuevo objeto al atributo de instancia `mr`, con el nro de filas y columnas de `m` y referencias a los mismos elementos.
- `xEscalar(r:Racional)` Multiplica cada elemento por `r`. Requiere `r` ligado y cada elemento de `mr` ligado.
- `obtenerElem(f,c:entero):Racional` Requiere `0<=f<obtenerNfil()` y `0<=c<obtenerNcol()`.
- `clone():MatrizRacional` Implementa clonado superficial.
- `equals(m:MatrizRacional):boolean` implementa igualdad superficial.
- `esRala():` retorna `true` si más de la mitad de los elementos representan el valor cero.
- `vectorMayores():Vector` Genera un objeto de clase `VectorRacional` en el cual el elemento correspondiente al índice `i`, es el mayor de la fila `i` de la matriz.
- `cantElem(r:Racional):entero` Cuenta la cantidad de elementos equivalentes a `r` en la matriz.

En la consulta `cantElem` la clase cliente no asume el compromiso de controlar que `r` esté ligado de modo que debe realizarse ese control antes de recorrer la matriz.

El planteo recursivo para decidir si una matriz de números racionales es la matriz identidad es ahora:

Caso trivial

Una matriz **mr** de 1×1 es la matriz identidad si su único elemento **mr**[0,0] representa a la unidad

Caso recursivo

Una matriz **mr** de $n \times n$ con $n > 1$ es la matriz identidad si

mr[i,n-1] representa a un cero para $0 \leq i < n-1$

mr[n-1,j] representa a un cero para $0 \leq j < n-1$

mr[n-1,n-1] representa a la unidad

y la matriz **mr'** de $(n-1) \times (n-1)$ es la matriz identidad

Este planteo es más general que el propuesto para una matriz de números reales.

Racional
<<Atributos de instancia>> numerador:entero denominador:entero
<<Constructor>> Racional(n,d:entero) <<Comandos>> establecerNumerador(n:entero) establecerDenominador(d:entero) <<Consultas>> obtenerNumerador():entero obtenerDenominador():entero equals(r:Racional):boolean suma(r:Racional):Racional resta(r:Racional):Racional producto(r:Racional):Racional cociente(r:Racional):Racional
El denominador es siempre positivo. Dos números racionales son iguales si representan un mismo valor.

- `Racional(n,d:entero)` Requiere `d` mayor a 0.
- `establecerDenominador(d:entero)` Requiere `d` mayor a 0.
- `equals(r:Racional):boolean` Computa `true` si el objeto que recibe el mensaje representa un número racional equivalente al parámetro.
- `suma(r:Racional):Racional` Requiere `r` ligado.
- `resta(r:Racional):Racional` Requiere `r` ligado.
- `producto(r:Racional):Racional` Requiere `r` ligado.
- `cociente(r:Racional):Racional` Requiere `r` ligado.

La implementación parcial de `Racional` es:

```
class Racional {
//El denominador es siempre mayor a 0
private int numerador,denominador;
public Racional(int n,int d){
//Requiere d > 0
    numerador = n;
    denominador = d;}
public int obtenerNumerador(){
    return numerador;}
```

```

public int obtenerDenominador(){
    return denominador;}
public boolean equals(Racional r){
    boolean ig = false;
    if (r!=null)
        ig = numerador*r.obtenerDenominador()==
            denominador*r.obtenerNumerador();
    return ig;}
public Racional suma (Racional r){
//Requiere r ligado
    int n = numerador*r.obtenerDenominador()+
        r.obtenerNumerador()* denominador;
    int d = denominador*r.obtenerDenominador();
    Racional s = new Racional(n,d);
    return s;}
public Racional producto (Racional r){
//Requiere r ligado
    return new Racional(numerador*r.obtenerNumerador(),
        denominador*r.obtenerDenominador());}
}

```

La implementación parcial de la clase `MatrizRacional` es:

```

class MatrizRacional {
/* Cada elemento de la matriz es una referencia ligada cuando se
ejecuta cualquier consulta, con excepción de existePos,
obtenerNFil, obtenerNCol y obtenerElem. Todos los servicios que
reciben a una matriz como parámetro requieren que la referencia
esté ligada. */
//Atributos de instancia
private Racional[][] mr;
//Constructor
public MatrizRacional(int nfil,int ncol){
//Requiere nfil y ncol mayores a 0
    mr=new Racional[nfil][ncol];}
//Comandos
public void establecerElemento (int f,int c,
                                Racional r) {
//Requiere f y c válidos
    mr[f][c]= r;}
public void establecerIdentidad () {
//Requiere que el número de filas sea igual al de columnas
    iniMatrizRacional();
    for (int j=0;j<obtenerNCol();j++)
        mr[j][j] = new Racional(1,1);    }
private void iniMatrizRacional () {
    for (int i=0;i<obtenerNFil();i++)
        for (int j=0;j<obtenerNCol();j++)
            mr[i][j] = new Racional(0,0);    }
public void xEscalar (Racional r) {
//Multiplica cada elemento por r, requiere r ligado
    for (int i=0;i<obtenerNFil();i++)
        for (int j=0;j<obtenerNCol();j++)
            mr[i][j] = mr[i][j].producto(r);}
public void copy (MatrizRacional m) {

```

```

/* Liga un nuevo objeto al atributo de instancia mr, con el nro de
filas y columnas de m y referencias a los mismos elementos.*/
mr = new Racional[m.obtenerNFil()][m.obtenerNCol()];
for (int i=0;i<obtenerNFil();i++) {
    for (int j=0;j<obtenerNCol();j++)
        mr[i][j] = m.obtenerElem(i,j); }
//Consultas
public int obtenerNFil ()      {
    return mr.length; }
public int obtenerNCol ()      {
    return mr[0].length; }
public boolean existePos(int f, int c) {
    return (f>=0 && f < obtenerNFil()) &&
        (c>=0 && c < obtenerNCol());}
public Racional obtenerElem(int f,int c){
//Requiere f y c válidos
    return mr[f][c];}
public int cantElem (Racional r) {
/*Cuenta la cantidad de elementos equivalentes a r en la matriz */
    int cant = 0;
    if (r != null)
        for (int i=0;i< obtenerNFil();i++)
            for (int j=0;j< obtenerNCol();j++)
                if (mr[i][j].equals(r)) cant++;
    return cant;}
public boolean esCuadrada () {
    return (obtenerNFil() == obtenerNCol()) ; }
public boolean esIdent () {
    if (!esCuadrada()) return false;
    else return esIdentidad(obtenerNFil());}
private boolean esIdentidad (int n) {
    if (n == 1) return (mr[0][0].equals(new Racional(1,1)));
    else return (mr[n-1][n-1].equals(new Racional(1,1)) &&
        esCeroFila(n-1) &&
        esCeroColumna(n-1)&&
        esIdentidad(n-1));}

```

Algunos servicios han reusado los algoritmos de la clase `Matriz` implementada antes, otros fueron adaptados pero el algoritmo responde a un mismo patrón. Dejamos pendiente como tarea completar la implementación. En la implementación de las consultas hemos considerado que, tal como establece el contrato, todas las referencias de `mr` están ligadas. Cuando la estructura puede procesarse aun si no todas las referencias están ligadas, es necesario establecer controles adicionales.

TDA y estructuras parcialmente ocupadas

La abstracción permite proponer soluciones similares para problemas en apariencia muy diferentes. Esas similitudes van a permitir reusar diseño y código, favoreciendo la productividad.

En los casos de estudio que presentamos a continuación se define una clase que encapsula una estructura homogénea, lineal, con acceso directo y parcialmente ocupada. Es decir, el constructor crea una estructura para mantener cierta cantidad máxima de componentes. Desde la clase cliente, se insertan y eliminan componentes de acuerdo a funcionalidad

especificada en el diseño. Toda la entrada y salida se hace desde la clase cliente de la clase que encapsula a la estructura de datos.

Utilizaremos el término **colección** para referirnos a una estructura con capacidad para mantener max elementos de un **tipo base**. En cada momento determinado sólo n de los max elementos referencian a un objeto del tipo base. Los n elementos ligados ocupan las primeras n posiciones de la estructura. De modo que las $max-n$ componentes nulas, están comprimidas en las últimas posiciones del arreglo. Si la posición de cada elemento en una colección no tiene relevancia, puede cambiar de posición siempre y cuando todas las n referencias ligadas se mantengan en las primeras n posiciones. Por ejemplo, si una colección mantiene el inventario de bienes de uso de una dependencia municipal y el orden en el cual se insertan los artículos en la colección es irrelevante, podrían cambiar de posición. En cambio, si una colección mantiene la secuencia de mensajes recibidos en un dispositivo, ordenados de acuerdo a como fueron recibidos e insertándose, no debería cambiarse la posición de cada mensaje porque dejarían de estar ordenados tal como fueron insertados.

Utilizaremos el término **tabla** para referirnos a una estructura con n elementos de un tipo base, cada uno de los cuales ocupa una posición que es significativa en la estructura. Por ejemplo, un micro se asigna a una unidad específica de un estacionamiento o un socio de un club tiene asignado un casillero particular que se identifica por su posición en la estructura que modela al conjunto de casilleros. Es decir las $max-n$ componentes nulas pueden estar intercaladas con las componentes ligadas.

Caso de Estudio: Correo

Una dependencia municipal desea resguardar una secuencia de mensajes recibidos en una cuenta de correo. Se plantea la necesidad de insertar nuevos mensajes en la secuencia, eliminar de acuerdo a algunos criterios determinados y procesar la secuencia para atender diferentes consultas y generar distintos reportes.

La secuencia es modelada mediante una colección en la cual los mensajes se insertan de acuerdo al orden en que se reciben. Es decir, la posición de cada elemento, queda determinada por el orden de inserción.

En el diagrama que sigue la clase **Correo** encapsula a un arreglo de elementos de clase **Mensaje** y un entero que representa la cantidad de mensajes, es decir la cantidad de elementos del arreglo que mantienen referencias ligadas. Las clases **Correo** y **Mensaje** están asociadas. Tal como se especifica en sus responsabilidades, la clase **Correo** se compromete a mantener las primeras **cant** posiciones del arreglo **T** ligadas y asegura que las demás posiciones no están ligadas.

Los elementos de la colección **Correo** pueden ser procesados, por ejemplo para calcular cuántos mensajes tienen más de n caracteres en el contenido, aun cuando la estructura no esté completa. Luego se insertan nuevos mensajes, se eliminan otros y se vuelve a procesar la estructura completa. La situación es diferente en otros problemas, como por ejemplo la estación meteorológica, en la cual se conoce la cantidad de elementos y la estructura está completa en el momento que se procesa.

Correo	Mensaje
<pre><<Atributos de instancia>> T:[] Mensaje cant:entero</pre>	<pre><<Atributos de instancia>> contacto:Contacto fecha:Fecha hora:Hora asunto:String contenido:String</pre>
<pre><< Constructor>> Correo(max:entero) <<Comandos>> insertar(m:Mensaje) eliminar(m:Mensaje) eliminar(c:Contacto) <<Consultas>> cantMen():entero estaLleno():entero hayMen():boolean mensaje(p:entero):Mensaje pertenece(m:Mensaje):boolean caracteres(n:entero):entero asunto(a:String):Correo dosenSec(c:Contacto):Mensaje</pre>	<pre><<Constructor>> Mensaje(c:Contacto,f:Fecha, h:Hora,a:String,t:String) <<Comandos>> <<Consultas>> obtenerContacto():Contacto obtenerFecha():Fecha obtenerHora():Hora obtenerAsunto():String obtenerContenido():String equals(m:Mensaje):boolean</pre>
Asegura que la primeras <code>cant</code> posiciones del arreglo <code>T</code> ligadas y las demás no están ligadas. El atributo <code>cant</code> mantiene cantidad de elementos ocupados en el arreglo y también la posición en la que se va a insertar el próximo elemento.	

- `Correo(max:entero)` crea un arreglo de `max` elementos, inicialmente nulos. Requiere `max` mayor a 0.
- `insertar(m:Mensaje)` Asigna el mensaje `m` a la posición `cant` y aumenta en 1 el valor de `cant`. Requiere que la clase cliente controle que la estructura no está llena, el parámetro `m` está ligado y no existe un mensaje con la misma identidad en la colección.
- `eliminar(m:Mensaje)` Busca el mensaje con la misma identidad que `m` y si existe, arrastra los que siguen una posición y actualiza `cant`. Si no existe el mensaje `m`, la colección no se modifica. Requiere `m` ligado.
- `eliminar(c:Contacto)` Elimina todos los mensajes del contacto que tiene la misma identidad que `c` y comprime los que quedan. Actualiza `cant`. Si no existen mensajes del contacto `c`, la colección no se modifica. Requiere `c` ligado.
- `pertenece(m:Mensaje):boolean` retorna `true` si existe un mensaje con la misma identidad que `m`. Requiere `m` ligado.
- `caracteres(n:entero):entero` computa la cantidad de mensajes con `n` caracteres en el contenido.
- `asunto(a:String):Correo` genera una estructura con los mensajes que corresponden al asunto `a`. Requiere `a` ligado.
- `dosenSec(c:Contacto):Mensaje` Dada una secuencia de mensajes $S = s_1, s_2, \dots, s_{cant-1}, s_{cant}$ la consulta `dosenSec` retorna s_k tal que s_k y s_{k-1} corresponden al mismo contacto `c`

y no existe un $j, j > k$, tal que s_j y s_{j-1} corresponden al mismo contacto c , con $0 \leq k < \text{cant}$ y $0 \leq j < \text{cant}$. Requiere c ligado.

La implementación parcial en Java de Correo es:

```
class Correo {
/* Asegura que la primeras cant posiciones del arreglo T ligadas y
las demás no están ligadas. El atributo cant mantiene cantidad de
elementos ocupados en el arreglo y también la posición en la que se
va a insertar el próximo elemento. */
//Atributos de Instancia
    private Mensaje[] T;
    private int cant;
//Constructor
/*Crea una estructura con capacidad para max mensajes*/
public Correo(int max) {
    T = new Mensaje [max];
    cant = 0;}
//Comandos
public void insertar(Mensaje m) {
/*Asigna el mensaje m a la primera posición libre del arreglo, es
decir, cant. Aumenta el valor de cant. Requiere que la clase cliente
controle que la estructura no está llena, el parámetro m está ligado y
no existe un mensaje con la misma identidad en la colección */
    T[cant++] = m;    }
public void eliminar( Mensaje m){
/* Busca un mensaje con la misma identidad que m en la secuencia de
mensajes, si lo encuentra arrastra los mensajes que siguen una
posición. Requiere m ligado.*/
    boolean esta = false; int i= 0;
    while (!esta && i<cantMen())
        if (T[i] == m)
            esta = true;
        else
            i++;
    if (esta) {
        cant--;
        arrastrar(i);}}
private void arrastrar(int i){
/* arrastra todos los elementos una posición hacia arriba*/
    while (i < cantMen()){
        T[i] = T[i+1];
        i++;}
    T[i]=null;}
//Consultas
public int  cantMen () {
    return cant;}
public boolean estaLleno() {
    return cant == T.length;}
public boolean hayMen() {
    return cant > 0;}
```

```

public Mensaje mensaje(int p){
/*Retorna el mensaje que corresponde a la posición p, si p no es una
posición válida retorna nulo*/
    Mensaje m = null;
    if (p>=0 && p < cantMen()) m = T[p];
    return m;}
public boolean pertenece (Mensaje m){
/*Decide si algún elemento de la colección tiene la misma identidad
que m. Requiere m ligado.*/
    boolean esta = false;
    for (int i = 0; !esta && i < cantMen() ; i++){
        esta = T[i] == m;}
    return esta;}
public int caracteres(int n){
/*Cuenta la cantidad de mensajes con más de n caracteres*/
    int car =0;
    String content;
    for (int i = 0; i < cantMen(); i++) {
        content = T[i].obtenerContenido();
        if (content.length() > n) car++;}
    return car;
}
public Correo asunto(String a){
/*Genera un objeto de clase Correo sólo con los objetos que
corresponden al asunto a. Requiere a ligado.*/
    Correo n = new Correo(cantMen());
    for (int i = 0; i<cantMen() ; i++)
        if (T[i].obtenerAsunto().equals(a)){
            n.insertar(T[i]);}
    return n;    } }

```

Las clases que usan los servicios provistos por `Correo` no conocen su representación interna. La clase `Correo` usa los servicios provistos por la clase `Mensaje`, sin conocer su implementación.

La clase tester debería verificar cada servicio considerando casos significativos. Por ejemplo, para verificar el servicio `eliminar`, los casos de prueba pueden ser: eliminar el primer mensaje, el segundo, el anteúltimo, el último. También hay que considerar que el elemento a eliminar no pertenezca a la colección. La verificación, como siempre, asume que la clase cliente cumple con sus responsabilidades.

Dado un método en la clase tester que incluye las siguientes declaraciones:

```

Mensaje sms;
Correo col;
sms = new Mensaje(...);
col = new Correo(1000);

```

En la clase cliente, cada mensaje `insertar` debería controlar previamente que el mensaje no fue insertado previamente, el parámetro real esté ligado y que la estructura no está llena:

```

if (sms != null && !col.pertenece(sms) && !col.estaLleno())
    col.insertar(sms);

```

El control de pertenencia requiere recorrer la estructura completa para retornar `false`, que probablemente sea el caso más frecuente.

Caso de Estudio: Inventario

Una dependencia municipal debe mantener un inventario permanente de los bienes de uso. Se plantea la necesidad de insertar nuevos artículos en el inventario, dar de baja a un artículo determinado y procesar el inventario para atender diferentes consultas y generar distintos reportes.

El inventario se modela mediante una colección en la cual la posición de los elementos no es relevante. El diagrama que sigue modela las clases asociadas `Inventario` y `Articulo`. La clase `Inventario` encapsula a un arreglo de elementos de clase `Articulo` y un entero que representa la cantidad actual de artículos en el inventario.

Inventario	Articulo
<pre><<Atributos de instancia>> T:[] Articulo cant:entero</pre>	<pre><<Atributos de instancia>> codigo:entero rubro:entero valor:real anio:entero</pre>
<pre><<Constructor>> Inventario(max:entero) <<Comandos>> alta(c:Articulo):boolean baja(c:Articulo) depreciarRubro(r:entero,p:real) <<Consultas>> cantArt():entero estaLleno():boolean recuperar(c:entero):Articulo pertenece(c:entero):boolean pertenece(c:Articulo):boolean unAnio(a:entero):Inventario</pre>	<pre><<Constructor>> Articulo(c:entero,r:entero, v:real,a:entero) <<Comandos>> depreciar(p:real) <<Consultas>> obtenerCodigo():entero obtenerRubro():entero obtenerValor():real obtenerAnio():entero equals(Articulo a):boolean</pre>
Asegura que las primeras <code>cant</code> posiciones del arreglo <code>T</code> están ligadas, las demás no están ligadas. El atributo <code>cant</code> mantiene cantidad de elementos ocupados en el arreglo y también la posición en la que se va a insertar el próximo elemento.	

- `Inventario(max:entero)` crea un arreglo de `max` elementos, inicialmente nulos. Requiere `max` mayor a 0.
- `alta(c:Articulo):boolean` si la estructura no está llena, asigna `c` a la posición `cant`, aumenta el valor de `cant` en 1 y retorna `true`, sino retorna `false`. Requiere que la clase cliente haya verificado que no existe un artículo con el mismo código que `c` en el inventario.
- `baja(art:Articulo)` Busca la posición un artículo equivalente a `art`, si existe y no es el último, asigna el último elemento del inventario a la posición encontrada y decrementa `cant`. Si existe y es la última posición, le asigna nulo y decrementa `cant`.

- `depreciarRubro(r:entero, p:real)` modifica el valor de cada artículo del rubro `r` decrementándolo de acuerdo al porcentaje `p`.
- `recuperar(c:entero):Articulo` Retorna el artículo con el código `c`.
- `pertenece(c:entero):boolean` Retorna `true` si un elemento de la colección tiene el código `c`.
- `pertenece(art:Articulo):boolean` retorna `true` si algún elemento de la colección es equivalente a `art`.
- `unAnio(a:entero):Inventario` Genera un nuevo inventario con los artículos que corresponden al año `a`.

El servicio baja no conserva el orden en el que se insertaron los artículos en el inventario.

La implementación en Java de `Inventario` es:

```
class Inventario {
/* Asegura que la primeras cant posiciones del arreglo T ligadas y
las demás no están ligadas. El atributo cant mantiene cantidad de
elementos ocupados en el arreglo y también la posición en la que se
va a insertar el próximo elemento.*/
//Atributos de Instancia
    private Articulo[] T;
    private int cant;
//Constructor
/*Crea una Coleccion con capacidad para max elementos. Requiere max
mayor a 0*/
    public Inventario(int max) {
        T= new Articulo [max];
        cant = 0;}
//Comandos
    public boolean alta(Articulo elem) {
/* Si la estructura no está llena, asigna c a la posición cant,
aumenta el valor de cant en 1 y retorna true, sino retorna false.
Requiere que la clase cliente haya verificado que no existe un
artículo con el mismo código que c en el inventario.*/
        boolean inserto=false;
        if (elem !=null && cant<T.length) {
            T[cant++] = elem;
            inserto = true;}
        return inserto;}
    public void baja( Articulo art){
/*Busca la posición un artículo equivalente a art, si existe y no es
el último, asigna el último elemento del inventario a la posición
encontrada y decrementa cant. Si existe y es la última posición, le
asigna nulo y decrementa cant*/
        boolean esta = false; int i= 0;
        while (!esta && i <  cantArt())
            if (T [i].equals(art)) esta = true;
            else i++;
        if (esta) {
            cant--;
            T[i] = T[cant];
            T [cant] = null;}}
```

```

public void depreciarRubro(int r, float p){
/*Modifica el valor de cada artículo del rubro r decrementándolo de
acuerdo al porcentaje p.*/
    float v;
    for (int i = 0; i < cantArt() ; i++)
        if(T[i].obtenerRubro() == r){
            v = T[i].obtenerValor();
            T[i].establecerValor(v*(1-p));} }
//Consultas
public int cantArt () {
    return cant;}
public boolean estaLleno() {
    return cant == T.length;}
public Artículo recuperar (int c){
/*Retorna el artículo con el código dado
boolean esta = false; int i;
for (i = 0; !esta && i < cantArt() ; i++){
    esta = T[i].obtenerCodigo() == c; }
if (esta) return T[i];
else return null;}
public boolean pertenece (int c){
/*Retorna true si un elemento de la Coleccion tiene el código c*/
boolean esta = false;
for (int i = 0; !esta && i < cantArt() ; i++)
    esta = T[i].obtenerCodigo() == c;
return esta;}
public boolean pertenece (Articulo art){
/*Retorna true si algún elemento de la colección es equivalente a
art.*/
boolean esta = false;
for (int i = 0; !esta && i < cantArt() ; i++)
    esta = T[i].equals(art);
return esta;}
public Inventario unAño(int a){
/* Genera un nuevo inventario con los artículos que corresponden al
año a. */
    Inventario n = new Inventario(cantArt());
    for (int i = 0; i < cantArt() ; i++)
        if(T[i].obtenerAño() == a)
            n.alta(T[i]);
    return n;}
}

```

Algunos servicios de `Inventario` tienen funcionalidades y responsabilidades similares a los que brinda la clase `Correo`, aunque cambian los nombres. Ambas clases modelan a una colección de elementos homogéneos y brindan un comando para agregar un elemento o eliminar un elemento.

Los comandos para insertar nuevos elementos tiene diferente responsabilidad en cada caso. El comando `alta` de la clase `Inventario` debe controlar que el parámetro no sea nulo para poder asegurar que todas las referencias ligadas ocupan las primeras `cant` posiciones el arreglo. Además, debe controlar que la estructura no está llena antes de asignar el nuevo artículo al inventario.

También los requisitos para los comandos eliminar son diferentes ya que en el caso del inventario el orden de los elementos no es relevante. Los servicios que realizan alguna forma de procesamiento son diferentes, porque dependen de la aplicación. Por supuesto podemos definir otras clases que compartan la funcionalidad de algunos servicios.

El siguiente caso de estudio define una clase que modela una tabla de elementos homogéneos. Mientras que las dos colecciones propuestas se caracterizan porque las componentes están comprimidas, utilizamos el nombre tabla para una estructura en la cual las componentes nulas y ligada pueden estar intercaladas.

Caso de Estudio: Parking

Una concesionaria dispone de un estacionamiento en el cual cada unidad de estacionamiento se identifica por un número. En un momento dado una unidad puede estar ocupada por un vehículo o libre. Cuando llega un vehículo se ocupa una unidad, cuando se retira se libera.

El estacionamiento puede ser modelado por una tabla, cada elemento representa a una unidad de estacionamiento. De modo que algunas posiciones de la tabla estarán ligadas a vehículos y otras no.

El siguiente diagrama modela el estacionamiento:

Parking	Vehiculo
<<Atributos de instancia>> T:[] Vehiculo	<<Atributos de instancia>> patente:String marca:String modelo:String km:int valuacion:real
<Cconstructor>> Parking (max:entero) <<comandos>> estacionar(v:Vehiculo,p:entero) estacionar(v:Vehiculo) salir(p:entero) salir(v:Vehiculo) <<Consultas>> maxUnidades():entero unidadesOcupadas():entero unidadOcupada(p:entero):boolean hayUnidadOcupada():boolean estaLleno():boolean estaVehiculo(v:Vehiculo):boolean existeUnidad(p:entero):boolean recuperaVehiculo(p:entero): Vehiculo valuacionTotal():real menorKm():Vehiculo cantModelo(mod:entero):entero	<<Constructor>> Vehiculo (p:String) <<Comandos>> establecerMarca(m:String) establecerModelo(m:String) establecerKm(k:int) establecerValuacion(v:real) <<Consultas>> obtenerPatente():String obtenerMarca():String obtenerModelo()String obtenerKm():int obtenerValuacion()real equals(v:Vehiculo):boolean menor(v:Vehiculo):boolean

- `Parking (max:entero)` Requiere `max` mayor a 0.
- `estacionar(v:Vehiculo, p:entero)` asigna el vehículo `v` a la unidad `p`, requiere que la unidad `p` exista y esté libre.

- `salir(v:Vehiculo)` busca un vehículo equivalente a `v` y si está, libera la unidad de estacionamiento asignada a dicho vehículo.
- `salir(p:entero)` libera la unidad de estacionamiento `p`. Requiere `0<=p<maxUnidades()`.
- `maxUnidades():entero` retorna la cantidad de unidades del estacionamiento.
- `unidadesOcupadas():entero` retorna la cantidad de unidades de estacionamiento ocupadas, es decir, asignadas a vehículos.
- `unidadOcupada(p:entero):boolean` retorna `true` si la unidad `p` está ocupada, requiere `0<=p<maxUnidades()`
- `hayUnidadOcupada():boolean` retorna `true` si hay al menos un vehículo en el estacionamiento, es decir, al menos una unidad ocupada.
- `estaLleno():boolean` retorna `true` si el estacionamiento no tiene unidades libres.
- `estaVehiculo(v:Vehiculo):boolean` si `v` está ligado y una unidad del estacionamiento mantiene un vehículo equivalente a `v`, retorna `true`, sino retorna `false`.
- `existeUnidad(p:entero):boolean` retorna `true` si `0<=p<maxUnidades()`
- `recuperarVehiculo(int p):Vehiculo` retorna el vehículo de la unidad `p`, requiere `0<=p<maxUnidades()`
- `valuacionTotal():real` retorna la valuación total de todos los vehículos que ocupan unidades en el estacionamiento.
- `menorKm():Vehiculo` retorna el vehículo con menor kilometraje, si hay varios con el mismo kilometraje, retorna el primero.
- `cantModelo(mod:entero):entero` retorna la cantidad de vehículos del modelo `mod`.

Las clases `Parking` y `Vehiculo` están asociadas. La clase `Parking` encapsula un arreglo de elementos de clase `Vehiculo`.

Si la clase cliente envía el mensaje estacionar a un objeto de clase `Parking` y no se cumple `0<=p<maxUnidades()`, se produce un error de ejecución. Si en cambio `p` es válido pero no corresponde a una unidad libre, se produce un error de aplicación.

La implementación en Java de `Parking` es:

```
class Parking{
//Atributo de instancia
private Vehiculo[] T;
//Constructor
public Parking(int max){
//Requiere max mayor a 0
    T = new Vehiculo[max];}
//Comandos
public void estacionar(Vehiculo v, int p) {
/*Asigna el vehiculo a la unidad, requiere que la unidad p exista y
esté libre*/
```

```
T[p] = v;}
public void salir(Vehiculo v){
/*Busca en el estacionamiento un vehículo equivalente a v y libera la
unidad*/
    int i = 0; boolean esta=false;
    while (i < T.length && !esta){
        if (T[i] != null)
            esta = v.equals(T[i]) ;
        else
            i++;}
    if (esta)
        T[i] = null;}
public void salir(int p){
/*Elimina el elemento de la unidad p, requiere que p sea un unidad
válida*/
    T[p] = null;}
//Consultas
public int maxUnidades(){
    return T.length;}
public int unidadesOcupadas(){
/*Retorna la cantidad de unidades en las que hay un vehículo
asignado*/
    int cant=0;
    for (int i=0; i<maxUnidades();i++)
        if (T[i] != null)
            cant++;
    return cant;}
public boolean unidadOcupada(int p){
//Requiere p válida
    return T[p] != null;}
public boolean hayUnidadOcupada(){
    int i = 0; boolean hay=false;
    while (i < T.length && !hay){
        if (T[i] != null)
            hay = true ;
        i++;}
    return hay;}
public boolean estaLleno(){
    int i = 0; boolean hay=false;
    while (i < T.length && !hay){
        if (T[i] == null)
            hay = true ;
        i++;}
    return !hay;}
public boolean estaVehiculo(Vehiculo v){
/*Decide si una unidad mantiene un vehículo equivalente a v que asume
ligado*/
    int i = 0; boolean esta=false;
    while (i < T.length && !esta){
        if (T[i] != null)
            esta = v.equals(T[i]) ;
        i++;}
    return esta;}
public boolean existeUnidad (int p) {
    return p >= 0 && p < T.length;}
```

```

public Vehiculo recuperarVehiculo(int p) {
//Requiere que p sea una posición válida
    return T[p];}
public Vehiculo menorKm() {
/*Asume que no hay vehículos con más de 500000 km
Puede retornar nulo si la tabla está vacía*/
    Vehiculo v=null; int menor = 500000;
    for (int i=0; i<maxUnidades();i++)
        if (T[i] != null && T[i].obtenerKm() < menor){
            v = T[i]; menor = v.obtenerKm();}
    return v;}
public int cantModelo(int mod) {
    int cant=0;
    for (int i=0; i<=T.length;i++)
        if (T[i] != null && T[i].obtenerModelo() == mod)
            cant++;
    return cant;}
public float valuacionTotal() {
    float v=0;
    for (int i=0; i<=T.length;i++)
        if (T[i] != null)
            v = v+T[i].obtenerValuacion();
    return v;}}

```

Los operandos no son conmutativos en la expresión:

```
(T[i] != null && T[i].obtenerModelo() == mod)
```

Y es necesario utilizar el operador con cortocircuito. Es decir, las siguientes expresiones computan error en ejecución si `T[i] == null`:

```

(T[i] != null & T[i].obtenerModelo() == mod)
(T[i].obtenerModelo() == mod && T[i] != null)

```

Notemos también que no se ha asignado a la clase `Parking` ni a sus clientes la responsabilidad de controlar que la unidad está disponible antes de que se ejecute el comando `estacionar`. Si la componente está ligada, la nueva referencia se sobrescribe sobre la anterior.

La clase `Parking` utiliza un arreglo parcialmente ocupado para representar una tabla de elementos cada uno de los cuales ocupa una posición particular. Podemos definir una clase similar para modelar el conjunto de casilleros asignados a los socios de un club, la asignación de mozos a las mesas de un bar o cualquier otro problema en el cual cada elemento se asigna a una posición específica.

En algunos de estos problemas cada elemento puede estar asignado a una única posición. Por ejemplo, un vehículo sólo estará asignado a una unidad del estacionamiento. En otros casos un mismo elemento puede estar ligado a varias posiciones de la tabla. Por ejemplo, un mismo mozo puede estar asignado a varias mesas.

El caso de estudio que sigue se modela mediante una colección, pero los elementos están ordenados de acuerdo a un atributo.

Caso de Estudio: Libreta de Contactos

Una aplicación permite administrar una libreta de contactos que mantiene el nombre, número de teléfono móvil, número de teléfono fijo y email de un conjunto de personas u organizaciones.

La clase `Libreta_Contactos` encapsula una colección de elementos de clase `Contacto`, representada con un arreglo parcialmente ocupado.

Libreta_Contactos	Contacto
<<Atributos de instancia>> T:[] Contacto cant:entero	<<Atributos de instancia>> nombre:String nroMovil:String nroFijo:String email:String
<< Constructor>> Libreta_Contactos(max:entero) <<Comandos>> insertar(con:Contacto) eliminar(con:Contacto) <<Consultas>> cantContactos():entero estaLleno():boolean pertenece(con:Contacto):boolean	<<Constructor>> Contacto (n:String) <<Comandos>> <<Consultas>> igualNombre(c:Contacto):boolean mayorNombre(c:Contacto):boolean
Los contactos se mantienen ordenados alfabéticamente por nombre y están comprimidos, de modo que desde la posición cant en adelante las referencias no están ligadas.	

- `insertar(con:Contacto)` Agrega `con` de modo tal que la colección se mantiene ordenada por nombre. Requiere que la estructura no esté llena, `con` ligado y no exista un contacto con el mismo nombre.
- `eliminar(con:Contacto)` Requiere `con` ligado. Elimina `con` de la colección. Si el contacto no existe el comando no modifica a la estructura.
- `pertenece(con:Contacto):boolean` retorna `true` si existe un contacto en la libreta con el mismo nombre que `con`.

Si la libreta de contactos se mantiene ordenada por nombre, el servicio `insertar` no puede implementarse asignando el nuevo contacto a la primera posición libre. Para comprender cómo implementar el servicio insertar comencemos visualizando la libreta de contactos a través de una figura:

Nombre	Número de Móvil	Número Fijo	Email

Consideremos que el primer contacto que se inserta corresponde a “Davini Laura”. Notemos que no interesan los otros atributos porque la libreta se ordena por nombre. Como la libreta está vacía el primer contacto ocupa la primera posición en la libreta:

Nombre	Número de Móvil	Número Fijo	Email
Davini Laura	...	...	...

Si luego vamos a insertar “Castro Ramón”, para que la Libreta quede ordenada, debemos *arrastrar* a “Davini Laura” desde la posición 0 a la posición 1 y luego asignar el nuevo contacto a la posición 0:

Nombre	Número de Móvil	Número Fijo	Email
Castro Ramón	...	...	...
Davini Laura	...	...	...

Si luego insertamos a “Álvarez María”, tenemos que arrastrar a los dos contactos anteriores una posición y luego asignar el nuevo contacto a 0:

Nombre	Número de Móvil	Número Fijo	Email
Álvarez María	...	...	...
Castro Ramón	...	...	...
Davini Laura	...	...	...

Si luego insertamos a “Funez María”, como no hay ningún contacto mayor al nuevo, insertamos directamente en la última posición:

Nombre	Número de Móvil	Número Fijo	Email
Álvarez María	...	...	...
Castro Ramón	...	...	...
Davini Laura	...	...	...
Funez María	...	...	...

Si luego insertamos un nuevo contacto con nombre “Cepeda Pedro”, arrastramos una posición a los dos contactos con nombres mayores al nuevo:

Nombre	Número de Móvil	Número Fijo	Email
Álvarez María	...	...	...
Castro Ramón	...	...	...
Cepeda Pedro	...	...	...
Davini Laura	...	...	...
Funez María	...	...	...

Cada figura nos permite visualizar la libreta de contactos de manera abstracta para mostrar cómo se inserta cada nuevo contacto. El diagrama de objetos, que también es una abstracción, es útil para graficar cómo se administra la memoria, pero no es una buena herramienta para diseñar el algoritmo insertar.

El algoritmo para el servicio insertar puede ser entonces:

Algoritmo insertar

Dato de Entrada: **nuevo**

 Buscar la posición del primer elemento mayor al **nuevo**

 Si existe

 Arrastrar todos los elementos desde esa posición hasta la última

 Asignar **nuevo** contacto a la posición encontrada

 Incrementar la cantidad de contactos

Para determinar la posición de inserción en una colección con `n` elementos, proponemos un planteo recursivo, que busca la posición del primer elemento mayor al nuevo contacto, puede ser:

Caso trivial: Si la colección está vacía la posición de inserción es 0
 Caso trivial: Si el nombre del último contacto de la colección es menor al buscado, la posición de inserción es `n`
 Caso Recursivo: Si el nombre del último contacto de la colección es mayor al buscado, busca la posición entre los primeros `n-1` contactos de la colección.

El valor inicial de `n` es `cant`, la colección no se modifica durante la búsqueda, pero al decrementarse `n` en 1, en cada ejecución se accede a una posición diferente. Si el nuevo elemento es mayor a todos, en particular si el nuevo elemento es el primero que se inserta, la posición de inserción es la primera libre, esto es `cant`.

El código del servicio `eliminar` también tiene que ser diseñado convenientemente, porque los elementos tienen que quedar ordenados y comprimidos. Si vamos a eliminar el contacto “Cepeda Pedro”, arrastramos una posición a los dos contactos con nombres mayores. En este caso la cantidad de contactos se decrementa en 1.

Nombre	Número de Móvil	Número Fijo	Email
Alvarez María	...	...	...
Castro Ramón	...	...	...
Davini Laura	...	...	...
Funez María	...	...	...

El algoritmo para el servicio `eliminar` puede ser entonces:

```
Algoritmo eliminar
Dato de Entrada: contacto
  Buscar la posición del contacto
  Si existe
    Arrastrar todos los elementos desde la última posición ligada
    hasta la posición que ocupa contacto que se va a eliminar
  Decrementar la cantidad de contactos
```

Para determinar la posición del elemento a eliminar en una colección con `cant` posiciones ligadas, proponemos un planteo recursivo:

Caso trivial: Si la colección está vacía el elemento no está y la posición del elemento a eliminar es `cant`
 Caso trivial: Si el último elemento de la colección es menor al buscado, el elemento no está y la posición del elemento a eliminar es `cant`
 Caso trivial: Si el último elemento de la colección es el buscado, la posición es `n-1`
 Caso Recursivo: Si el último elemento de la colección es mayor al buscado, busca la posición en los primeros `n-1` elementos de la colección.

El valor inicial de `n` es el valor del atributo de instancia `cant`. La implementación en Java puede ser entonces:

```
class Libreta_Contactos{
```

```
/*Los contactos se mantienen ordenados alfabéticamente por nombre y
están comprimidos, de modo que desde la posición cant en adelante las
referencias no están ligadas.*/
//Atributos de instancia
private Contacto [] T;
private int cant;
//Constructor
public Libreta_Contactos(int max){
    T = new Contacto [max];}
//Comandos
public void insertar (Contacto nuevo){
    /* Agrega el contacto nuevo de modo tal que la colección se mantiene
ordenada por nombre. Requiere que la estructura no esté llena, el
parámetro nuevo ligado y no exista un contacto con el mismo nombre.
*/
    int pos = posInsercion(nuevo,cant);
    arrastrarDsp (pos,cant-pos);
    T[pos] = nuevo;
    cant++; }
private void arrastrarDsp (int pos,int n){
    if (n > 0){
        T[pos+n] = T[pos+n-1];
        arrastrarDsp(pos,--n);    } }
public void eliminar(Contacto con){
    /* Elimina con de modo tal que la colección se mantiene ordenada por
nombre. Requiere con ligado. Si el contacto no existe el comando no
modifica a la estructura. */
    int pos = posElemento (con,cant);
    if (pos < cant){
        arrastrarAnt(pos,cant-pos-1);
        cant--;
        T[cant] = null; } }
private void arrastrarAnt(int pos,int n){
    if (n > 0){
        T[pos] = T[pos+1];
        arrastrarAnt(++pos,--n);    }}
//Consultas
public int cantContactos (){
    return cant;}
public boolean estaLleno (){
    return cant == T.length;}
public boolean pertenece(Contacto con){
    return con != null && posElemento(con,cant) < cant;}
private int posInsercion (Contacto con,int n){
    /* Si existen elementos con nombres mayores a los de con, retorna la
posición del primero, sino retorna 0*/
    int pos = 0;
    if (n > 0)
        if (con.mayorNombre(T[n-1].obtenerNombre()) ||
            con.igualNombre(T[n-1].obtenerNombre()))
            pos = n;
        else
            pos = posInsercion (con,--n);
    return pos; }
```

```
private int posElemento(Contacto con,int n){
/* si existe un contacto con el mismo nombre que con, retorna la su
posición, sino retorna cant*/
    int pos=cant;
    if (n > 0)
        if (T[n-1].igualNombre(con.obtenerNombre()))
            pos = n-1;
        else
            if (T[n-1].mayorNombre(con.obtenerNombre()))
                pos = posElemento (con,--n);
    return pos;}
}
```

El arreglo mantiene una colección homogénea de elementos. La colección se crea con capacidad para mantener **max** elementos pero inicialmente está vacía. En la colección se puede insertar un elemento en forma ordenada, eliminar un elemento manteniendo el orden y decidir si un elemento pertenece a la colección.

La implementación brinda dos métodos privados. La consulta **posInsercion** se usa desde el comando **insertar**, retorna la posición que sigue al último elemento menor al contacto **con**. La consulta **posElemento** se usa tanto en el comando **insertar** como en la consulta **pertenece**, si algún contacto referenciado en la estructura tiene el mismo nombre que **con**, retorna el subíndice en el arreglo, en caso contrario retorna **cant**.

Mantener la estructura ordenada tiene un costo en ejecución, cada inserción requiere buscar la posición y arrastrar, en promedio, la mitad de los elementos. Como contrapartida las búsquedas son más eficientes, ya que si se busca un elemento y éste no pertenece a la colección, no es necesario recorrerla toda. Si las búsquedas son más frecuentes que las inserciones, mantener la estructura ordenada puede ser una decisión acertada.

La eficiencia del algoritmo insertar puede mejorar si al mismo tiempo se busca la posición de inserción y se arrastran los elementos. Esta estrategia es posible dado que la clase cliente controla que hay al menos una posición libre y no existe un elemento con el mismo nombre que el nombre del contacto que se agrega.

Algoritmo insertar

Dato de entrada: **nuevo**

Buscar desde la última posición **cant-1** hasta 0, la posición **pos** del primer elemento menor al **nuevo** y arrastrar cada elemento de la posición **i** a la posición **i+1**

Si existe Asignar **nuevo** contacto en **pos+1**

Sino Asignar **nuevo** contacto en **cant**

Incrementar **cant**

Si la responsabilidad de controlar que no haya dos contactos con el mismo nombre recae en la clase cliente, es necesario recorrer dos veces la estructura, aunque sea parcialmente, para insertar un nuevo elemento. La primera cuando la clase cliente envía el mensaje **pertenece**, la segunda cuando el comando **insertar** busca la posición de inserción. Una alternativa más eficiente es que la clase **Libreta_Contactos** asuma la responsabilidad de controlar que no haya repetidos y el comando **insertar** busque el contacto, si existe no lo inserta, si no existe, encontró un contacto con nombre mayor al parámetro y esa es la posición de inserción.

Cuando una colección de elementos está ordenada de acuerdo a un atributo y es necesario decidir si un elemento en particular pertenece a la colección, es posible aplicar una estrategia conocida como **búsqueda binaria**. La estrategia consiste en partir la estructura en mitades, considerando que el **elemento buscado** puede ser:

- Igual al que está en el medio
- Menor que el que está en el medio
- Mayor que el que está en el medio

De modo que podemos proponer un algoritmo recursivo:

```
Algoritmo Búsqueda Binaria
Dato de entrada: elem
  si el elemento que está en el medio es el buscado
    EXISTE
  si hay un sólo elemento y no es el buscado
    NO EXISTE
  sino
    si el elemento que está en el medio es menor al buscado
      Descartar la primera mitad
      Buscar en la segunda mitad
    sino
      Descartar la segunda mitad
      Buscar en la primera mitad
```

Para nuestro caso de estudio específico podemos refinar el algoritmo:

```
Algoritmo Búsqueda Binaria
Dato de entrada: contacto
  si el contacto que está en el medio de la colección
    es el buscado
      EXISTE
  si hay un sólo contacto y no es el buscado
    NO EXISTE
  sino
    si el contacto que está en el medio es menor al buscado
      Descartar la primera mitad de la Libreta de contactos
      Buscar en la segunda mitad de la Libreta de contactos
    sino
      Descartar la segunda mitad de la Libreta de contactos
      Buscar en la primera mitad de la Libreta de contactos
```

El algoritmo puede refinarse todavía más considerando que los datos están en un arreglo:

```
Algoritmo BúsquedaBinaria
Dato de entrada: ini,fin,contacto
mitad ← (ini+fin)/2
si Tmitad tiene el mismo nombre que el contacto
  EXISTE
sino
  si ini ≥ fin
    NO EXISTE
  sino
    si Tmitad tiene el nombre menor a contacto
      BuscarBinaria mitad+1,fin,contacto
    sino
      BusquedaBinaria ini,mitad-1,contacto
```

Aunque esta versión sigue siendo abstracta, la traducción a un lenguaje de programación es más directa que en el caso anterior. La recursividad permite modelar naturalmente esta estrategia de búsqueda, justamente porque está especificada en forma recursiva.

La estrategia también puede modelarse a través de un algoritmo iterativo:

```
Algoritmo BúsquedaBinaria
Dato de entrada: contacto
ini ← 1
fin ← cantidad de contactos
mitad ← (ini+fin)/2
existe ← falso
mientras no existe y ini ≤ fin
    si Tmitad tiene el nombre igual a contacto
        existe ← verdadero
    sino
        si Tmitad tiene el nombre menor a contacto
            ini ← mitad+1
        sino
            fin ← mitad-1
```

El algoritmo es generalizable a cualquier estructura de datos lineal y ordenada, siempre que sus elementos puedan accederse directamente de acuerdo a su posición.